

Collection Class II

Array

The Arrays class in the java.util package is a part of the Java Collection Framework. This class provides static methods to dynamically create and access Java arrays. The length of an array is established when the array is created. After creation, its length is fixed.

Various Methods to Define an ArrayList:

```
int arr1[] = { 3 , 1 , 5, 2, 9};  
int[] arr2 = new int[4];
```

Size of Array:

```
int sz1 = arr1.length;  
int sz2 = arr2.length;
```

Sort the array;

```
// sort the array
Arrays.sort(arr1);
System.out.println(Arrays.toString(arr1));
```

Compare two array:

```
// compare 2 Arrays
System.out.println(Arrays.compare(arr1, arr2));
System.out.println(Arrays.equals(arr1, arr2));
```

Fill the specific value

```
// fill with specific value
Arrays.fill(arr2, 5);
System.out.println(Arrays.toString(arr2))
```

Comparator

Method of comparator functions:

```
class SortInt implements Comparator<Integer>{
    @Override
    public int compare(Integer a, Integer b){
        return b-a;
    }
}
```

```
Integer[] arr = {5, 1, 7 , 9 , 10 , 2};
```

```
//Method 1 : creating a class and Override compare function
Arrays.sort(arr, new SortInt());
```

```
//Method 2 : Override compare function as a inline function
Arrays.sort(arr, new Comparator<Integer>(){
    @Override
    public int compare(Integer a, Integer b){
        return b-a;
    }
});

//Method 3 : using Collections framework reverseOrder function
Arrays.sort(arr, Collections.reverseOrder());

//Method 4 : Using Lambda Function
Arrays.sort(arr , (o1 , o2) -> o2 - o1 );

// for(int i : arr) System.out.print(i+" ");
```

String

String is basically an object that represents a sequence of char values. Whenever a String Object is created as a literal, the object will be created in the String constant pool. This allows JVM to optimize the initialization of String literal.

Creating a String and initializing:

```
char[] ch={'c','o','d','i','n','g','n','i','n','j','a'};
String str1=new String(ch);

String str2="codingNinja";
    // The string can also be declared using a new operator i.e.
dynamically allocated.
    // In case String are dynamically allocated they are assigned a new
```

memory location in heap.

```
// This string will not be added to the String constant pool.  
String str3=new String("ninjaCoding");
```

Useful operations:

```
// substring of String  
String substr = str1.substring(0,4);  
System.out.println(substr);  
  
// concat 2 String  
String concatString = str1.concat(str2);  
System.out.println(concatString);  
  
// Convert String to Lower case  
String str4 = "CODINGNINJA";  
String str5 = str4.toLowerCase();  
System.out.println(str5);  
  
// can read about toUpperCase , trim , equals, etc
```

length of the string:

```
//Length of String  
int sz = str1.length();
```

Accessing the ith element of the queue:

```
// Accessing ith character of String  
for(int i=0;i<sz;i++) System.out.print(str1.charAt(i)+" ");
```

String builder:

The Java String is immutable which means it cannot be changed. Whenever we

change any string, a new instance is created. The **StringBuilder** in Java represents a mutable sequence of characters. Since the String Class in Java creates an immutable sequence of characters, the StringBuilder class provides an alternative to String Class, as it creates a mutable sequence of characters.

Changing the i^{th} character of the string:

```
StringBuilder str6 = new StringBuilder("CoddddingNinja");
System.out.println(str6);

// Changing ith character in string
str6.setCharAt(0, 'x');
System.out.println(str6);
```

Reverse a string :

```
// reverse a string , but these functions return stringbuilder and thus
convert them
String revString = str6.reverse().toString();
System.out.println(revString);
```

Deleting an element int the string

```
String str7 = str6.delete(0,1).toString();
```

Pairs

Pair is basically a combination of two values which can be of different data types. We can find the *Pair* class in the *javafx.util* package. The constructor of this class takes two arguments, a key and its corresponding value:

Creating a pair:

```
Pair<Integer, String> pair = new Pair<>(1, "One");
Integer key = pair.getKey();
String value = pair.getValue();
```

Implementation of pair:

```
public class Pair<T>{
    T p1, p2;
    void setValue(T a, T b){
        this.p1 = a;
        this.p2 = b;
    }
    boolean equals(Pair a){
        if(this.p1 == a.p1 && this.p2 == a.p2) return true;
        return false;
    }
}
```

First and the last position of an element in a sorted array.

Problem Statement:

You are given a sorted array **ARR** consisting of **N** integers and an integer **X**. You need to find the first and last position of occurrence of **X** in the array.

Approach:

The given input array is sorted, thus we can apply binary search to the array. To apply binary search we can also use the predefined function present in the collection class(**indexOf**). Let's see both the approaches.

Binary search:

Binary search prunes the search space by a factor of 2, in every step. Thus, we choose a mid-index to compare the elements in the current range [**LO**, **HI**]. The maximum depth of the binary search call stack is **O(log(N))**.

- Initialize the search space as **LO = 0** and **HI = N-1**.
 - To calculate the first occurrence, two conditions need to be checked at mid-value:
 - If **MID = 0**, that means it's the leftmost value in the array and no element lies to its left. Or if the value at mid position equals **X**, and the value at the **MID-1** index is not equal to **X**. Then, **MID** is the first occurrence of **X** in the array, return **MID**.
 - Else if **X > ARR[MID]**, that means **X** will lie in **[MID+1, HI]** indices of the array.
 - Else **X** will lie in **[LO, MID-1]** indices of the array.
- To find the first occurrence of the value **X** in the sorted array, use the binary search :
 - Set **LO = 0, HI = N-1**.
 - Exit condition for Search function: **LO <= HI**
 - For any **MID** value , where **MID = (LO + HI)/2**:
 - a. If **MID** equals to 0 or if **ARR[MID] = X**, and **X > ARR[MID-1]**, return **MID**.
 - b. Else if, **X > ARR[MID]**, then set **LO = MID+1**.
 - c. Else **X < ARR[MID]**, then set **HI = MID-1**.
 - Else return **-1**.
- To find the last occurrence of **X**, two conditions need to be checked at mid-value:

- If **MID = N-1**, that is if mid is at last position or if **ARR[MID]** equals **X**, and element at next index is higher i.e. **ARR[MID+1] > X**, which implies that **MID** is the last occurrence of element X.
 - Else if **X > ARR[MID]**, that means **X** will lie in **[MID+1, HI]** indices of the array.
 - Else **X** will lie in **[LO, MID-1]** indices of the array.
-
- To find the last occurrence of the value **X** in the sorted array, use the binary search :
 - Set **LO = 0, HI = N-1**.
 - Exit condition for Search function: **LO <= HI**
 - For any MID value , where **MID = (LO + HI)/2** :
 - a. If MID equals to **N-1** or if **ARR[MID] = X**, and **X < ARR[MID+1]**, return **MID**.
 - b. Else if, **X < ARR[MID]**, then set **LO = MID-1**.
 - c. Else **X > ARR[MID]**, then set **HI = MID+1**.
 - Else return -1.

Collections Class:

There are two very useful functions present in the collections class which are based on binary search :

1. indexOf:- It returns first occurrence index of element x in ArrayList.
2. lastIndexOf:- It returns last occurrence index of element x in ArrayList.

We can use the above two functions directly to solve the problem.

Code:

```
import java.io.*;
import java.util.*;

class CodingNinja {

    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int val = sc.nextInt();
        ArrayList<Integer> arr = new ArrayList<Integer>();
        for(int i=0;i<n;i++){
            arr.add(sc.nextInt());
        }

        // first occurrence in the ArrayList
        // indexOf return first occurrence index of element x in ArrayList
        int firstOcc = arr.indexOf(val);
        // lastIndexOf return last occurrence index of element x in
        ArrayList
        int lastOcc = arr.lastIndexOf(val);

        System.out.println("First Occurance of "+val+" is "+firstOcc);
        System.out.println("Last Occurance of "+val+" is "+lastOcc);
    }
}
```

Time Complexity:

$O(\log(N))$, where N is the size of the sorted array.

Since here we are using binary search, therefore the time complexity grows by order of $O(\log(N))$.

Space Complexity:

$O(1)$, as we are using constant extra memory.

Time-based key-value pair

Problem Statement:

You are given a stream of tuples which constitute three things '**Key**', '**Val**', and the '**Timestamp**'.

Your task is to implement the '**time-based**' class having the two functions:

1. The first function is 'setKey(string Key, string Val, int Timestamp)', which stores the 'Key' and the 'Val' along with the 'Timestamp'.
2. The second function is 'getValue(string TargetKey, int TargetTimestamp)', which returns the value of 'Val' associated with the 'TargetKey' such that its 'Timestamp' value is less than or equal to the 'TargetTimestamp'. If there are multiple values of 'Val', return the value of 'Val' with the highest value of 'Timestamp' among the valid ones. If there is no valid value of 'Val' return "-1" as a string.

'Timestamps' will always be in strictly increasing order.

Approach:

- Create a class '**Node**' with two parameters i.e value which will store the value of input '**value**' and '**timestamp**' which will store the value of input 'TIMESTAMP'.
- Create a hashmap keyTimestampVal, which will store '**key**' and an array of objects of class '**Node**' as an entry in a map.

- When the **'setter(key, value, timestamp)'** function is called, insert **'key'** as key and value as an object of class **'Node'** in our hashmap.
- Now when the **'getter(targetKey, targetTimeStamp)'** function is called, then perform the following steps:
 - First, check if the value of the timestamp of the first entry of hashmap at that **'targetKey'** is greater than the value of **'targetTimeStamp'** then return **-1** as string because it means there does not exist any value of timestamp whose value is less than **'targetTimeStamp'**.
 - Now check if the value **'targetTimeStamp'** is greater than or equal to the value of the timestamp of the last entry of hashmap at that **'Key'** then return the value of **'value'** of the last entry in hashmap. This is because it is the largest valid value of **'value'** as per our condition which is our answer.

Now we can find the key using contain function which is based on binary search and works in **$O(\log(n))$** .

Code:

```
import java.io.*;
import java.util.*;

class Node{
    String value;
    int timestamp;
    Node(String value, int timestamp) {
        this.value = value;
        this.timestamp = timestamp;
    }
}
```

```
    }  
}  
  
class CodingNinja {  
    public static void setter(String key, String value, int  
timestamp, Map<String, TreeSet<Node>> mp){  
        TreeSet<Node> pq;  
        if (mp.containsKey(key)) {  
            mp.get(key).add(new Node(value, timestamp));  
        }  
        else{  
            pq = new TreeSet<>((a, b) -> b.timestamp - a.timestamp);  
            pq.add(new Node(value, timestamp));  
            mp.put(key, pq);  
        }  
    }  
  
    public static String getter(String key, int timestamp, Map<String,  
TreeSet<Node>> mp){  
        if (!mp.containsKey(key)) return "";  
  
        TreeSet<Node> pq = mp.get(key);  
  
        for (Node item: pq) {  
            if (item.timestamp == timestamp) {  
                return item.value;  
            } else if (item.timestamp <= timestamp) {  
                return item.value;  
            }  
        }  
        return "";  
    }  
  
    public static void main(String[] args){  
  
        Map<String, TreeSet<Node>> mp = new HashMap<>();  
  
        setter("coding", "ninja", 1, mp);  
        setter("coding", "coder", 4, mp);  
    }  
}
```

```
setter("coding", "learning", 10, mp);

setter("Competitive", "Coding", 3, mp);
setter("Competitive", "learning", 5, mp);
System.out.println("Case#1 "+getter("coding", 0, mp));
System.out.println("Case#2 "+getter("coding", 2, mp));
System.out.println("Case#3 "+getter("coding", 4, mp));
System.out.println("Case#4 "+getter("coding", 9, mp));
System.out.println();
System.out.println("Case#5 "+getter("Competitive", 1, mp));
System.out.println("Case#5 "+getter("Competitive", 4, mp));
System.out.println("Case#5 "+getter("Competitive", 10, mp));
}
}
```

Time Complexity: $O(N \cdot \log(N))$, where 'N' is the total number of queries.

Since there are **N** queries so when the '**setter()**' function is called, then insertion operation takes **$O(1)$** time per query, and when the '**getter()**' function is called, then binary search takes **$O(\log(N))$** time per query which in takes a total of **$O(N \cdot \log(N))$** time. So overall time complexity will be **$O(N \cdot \log(N))$** .

Space Complexity:

$O(N)$, where 'N' is the total number of queries.

We created a hashmap which uses **$O(N)$** space. So overall space complexity will be **$O(N)$** .